

**Міністерство освіти і науки України
Державний університет інформаційно-комунікаційних технологій
(ДУІКТ)**

**Факультет інформаційно-комунікаційних технологій
Кафедра Комп'ютерні науки**

**МЕТОДИЧНІ ВКАЗІВКИ
до виконання лабораторної роботи № 7**

з дисципліни «Методи та засоби штучного інтелекту»

**Тема: Дослідження генетичних алгоритмів на задачі пошуку
екстремумів функції засобами Python**

Спеціальність: 122 - Комп'ютерні науки

Програмне забезпечення:
https://github.com/fleceen1/Methods_and_features_of_piece_ntelligence_lab_7

м. Київ, 2026

Лабораторна робота 7

Державний університет інформаційно-комунікаційних технологій (ДУІКТ), м. Київ. Практична робота виконується у Python за готовим кодом з GitHub.

Тема: Дослідження генетичних алгоритмів на задачі пошуку екстремумів функції за допомогою засобів Python.

Мета

Вивчити основні принципи генетичного алгоритму та придбати навички оптимізації функцій за допомогою генетичних алгоритмів засобами Python.

Завдання

1. Створити й виконати сценарій оптимізації функції згідно з варіантом (табл. 8.1).
2. Побудувати графік функції.
3. Оптимізувати функцію з програмним керуванням параметрами ГА.
4. Оптимізувати функцію з використанням графічного інтерфейсу (GUI).

Таблиця 8.1 - Функції для оптимізації

№ з/п	Функція	Інтервал	Знайти
1	$y(x) = 3x^3 - 4x^2 + 2x - 8$	$[-6; -2]$	максимум
2	$y(x) = -6/(x-3) + 2/(x-1) + 8$	$[-4; 8]$	мінімум
3	$z(x,y) = x^2 + y^2 - x \cdot y \cdot e^{-(x+y)}$	$x, y \in [-2; 2]$	мінімум
4	та сама $y(x)$, що в п. 2	$[-4; 8]$	максимум
5	та сама $z(x,y)$, що в п. 3	$x, y \in [-2; 2]$	максимум

Примітка. Для варіантів 2 і 4 функція має полюси при $x = 1$ та $x = 3$. У програмному проєкті для стабільності оптимізації додано штраф у малих околах полюсів - про це слід зазначити у звіті.

Зміст звіту

1. Титульна сторінка.
2. Мета роботи.
3. Завдання.
4. Сценарій оптимізації (команда `main.py` або фрагмент поля «Результат» у GUI).
5. Опис виконання по пунктах завдання (хід роботи) із скріншотами.
6. Таблиця результатів експерименту (завдання 3).
7. Висновки.
8. Відповіді на контрольні питання.

Контрольні питання

1. Перерахуйте основні особливості ГА.
2. Перелічіть генетичні оператори.
3. Які критерії зупинки використовуються для ГА?
4. Опишіть схему класичного ГА.
5. У чому полягають особливості спільного використання генетичних операторів?

Теоретичні відомості

8.1 Основи генетичних алгоритмів

Генетичні алгоритми (Genetic Algorithms, ГА) є складовою еволюційних методів, оскільки виникли в результаті спроб копіювання еволюції живих організмів.

Генетичні алгоритми - це процедури пошуку, засновані на механізмах природного відбору і спадкування; у ГА використовується принцип виживання найбільш пристосованих індивідів. ГА мають певні переваги в порівнянні з традиційними методами оптимізації, оскільки поєднують кращі властивості градієнтних методів та методів евристичного пошуку.

Уявімо штучний світ, населений кількістю N індивідів (особин), причому генетичний код кожного індивіду - це деякий розв'язок нашої задачі. Разом усі індивіди утворюють популяцію P_0 . Для простоти вважається, що генотип кожного індивіда записується у вигляді однієї хромосоми X . У ГА хромосома X індивіда є впорядкованою послідовністю з M генів, тобто $X = \{G_1, G_2, \dots, G_M\}$ - числовий вектор, який описує розв'язок задачі.

Для утворення нових хромосом і пошуку серед них найкращих використовуються оператори: схрещування - об'єднання частин хромосом-батьків; мутації - випадкової зміни окремих генів; селекції - вибору хромосом для наступної ітерації алгоритму.

Основні відмінності ГА від традиційних задач оптимізації такі:

1. виконують пошук розв'язку не з однієї точки, а з деякої популяції;
2. використовують тільки цільову функцію, а не її похідні або іншу додаткову інформацію;
3. використовують ймовірнісні, а не детерміновані правила відбору.

Терміни ГА

- **Популяція** - кінцева множина особин.
- **Хромосома** - впорядкована послідовність генів (кодовий рядок, вектор параметрів).
- **Ген** - атомарний елемент генотипу.
- **Генотип** - набір хромосом даного індивіду.
- **Фенотип** - набір значень, що відповідають даному генотипу.
- **Алель** - значення конкретного гена.
- **Локус** - позиція гена в хромосомі.
- **Покоління** - чергова популяція в генетичному алгоритмі.
- **Функція пристосованості (fitness)** - міра пристосованості індивіда. Задача оптимізації зводиться до пошуку найбільш пристосованого індивіда.

Методи селекції хромосом

Метод рулетки. У методі рулетки кожній хромосомі X_i ставиться у відповідність сектор колеса рулетки, величина якого пропорційна функції пристосованості F даної хромосоми. Ймовірність селекції хромосоми X_i дорівнює $p_s(x_i) = F(x_i) / \sum F(x_j)$.

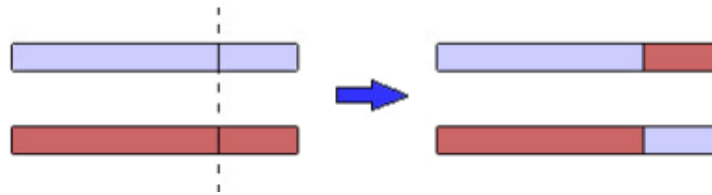
Рангова селекція. Індивіди популяції впорядковуються за значенням $fitness$; кращим присвоюються менші ранги, ймовірність вибору залежить від рангу.

Турнірна селекція. З популяції випадковим способом вибирається t хромосом, краща з яких (переможець) стає батьком. Розмір туру $2 \leq t < N$, зазвичай $t = 2$. У проєкті Python використовується саме турнірна селекція (за замовчуванням $t = 3$).

Генетичні оператори

Оператор схрещування (кросовер). Вибираються пари хромосом із батьківської популяції; відбувається обмін фрагментами між двома батьківськими хромосомами. У класичному бінарному ГА точка схрещування вибирається випадково. У *real-coded* ГА часто застосовують арифметичний кросовер: нащадок $= \alpha \cdot \text{батько}_1 + (1 - \alpha) \cdot \text{батько}_2$.

Рисунок 8.1 - Умовна схема схрещування



Оператор мутації з ймовірністю p_m змінює значення гена. У ГА мутація, ймовірність якої зазвичай невелика ($p_m < 0.2$), допомагає «вибити» популяцію з локального екстремуму та перешкоджає передчасній збіжності. У проєкті Python застосовується гаусова мутація з відсіканням до меж `bounds`.

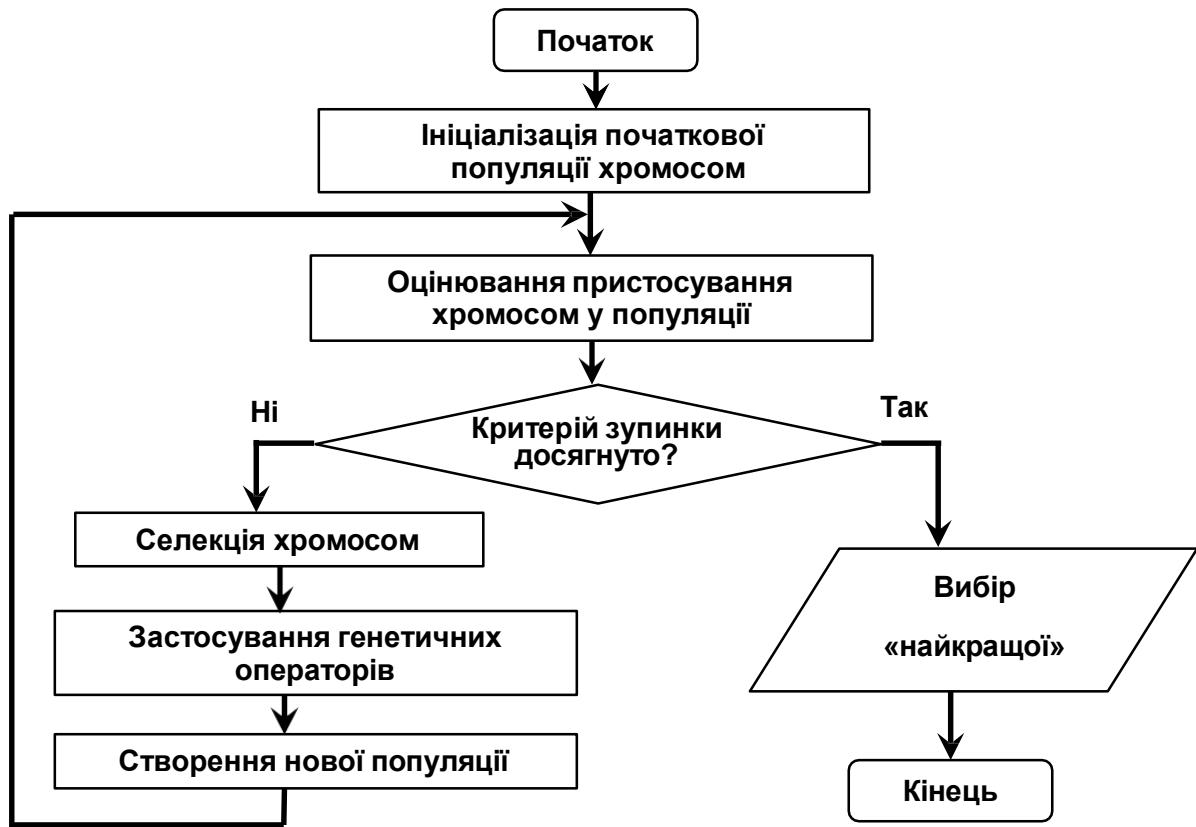
Класичний генетичний алгоритм

У класичному ГА кожний ген записується у вигляді набору бітів (бінарне кодування). У сучасних `real-coded` ГА хромосома - це безпосередньо вектор дійсних чисел у заданих межах (як у файлі `genetic_algorithm.py` цього проєкту).

Основний принцип роботи ГА можна описати кроками:

5. Генеруємо початкову популяцію з n хромосом.
6. Обчислюємо для кожної хромосоми її придатність (`fitness / cost`).
7. Вибираємо пару хромосом-батьків за допомогою селекції.
8. Проводимо схрещування двох батьків із ймовірністю p_c , створюючи нащадків.
9. Проводимо мутацію нащадків із ймовірністю p_m .
10. Повторюємо кроки 3–5, доки не буде сформовано нове покоління.
11. Повторюємо кроки 2–6, доки не буде досягнуто критерій закінчення.

Рисунок 8.2 - Блок-схема класичного генетичного алгоритму



Основними параметрами ГА є:

- ймовірність мутації p_m ;
- ймовірність схрещування p_c ;
- розмір популяції N ;
- кількість поколінь (ітерацій);
- метод селекції;
- критерії зупинки (ліміт поколінь, час, поріг якості, відсутність покращення).

Критерії зупинки алгоритму

- Generations - алгоритм зупиняється, коли кількість поколінь досягає заданого значення.
- TimeLimit - зупинка після заданого часу (сек).
- FitnessLimit - зупинка, коли найкраще значення fitness досягає порогу.
- StallGenLimit - зупинка при відсутності поліпшення протягом кількох поколінь.

- TolFun - зупинка при малій відносній зміні найкращого fitness.

У проєкті Python за замовчуванням використовується зупинка за кількістю поколінь (параметр `--generations` у `main.py` або «Покоління» у GUI).

8.2 Генетичний алгоритм у Python

Параметри ГА можна задавати через командний рядок (main.py) або через графічний інтерфейс (gui.py). Програмний проєкт розміщено у репозиторії GitHub (https://github.com/flece1/Methods_and_features_of_piece_ntelligence_lab_7).

Студент клонує репозиторій, встановлює залежності та виконує чотири завдання лабораторної роботи.

8.2.0 Підготовка до роботи

Крок 1. Отримати проєкт з GitHub:

```
cd %USERPROFILE%\Desktop  
git clone  
https://github.com/flece1/Methods\_and\_features\_of\_p  
iece\_ntelligence\_lab\_7.git  
cd Methods_and_features_of_piece_ntelligence_lab_7
```

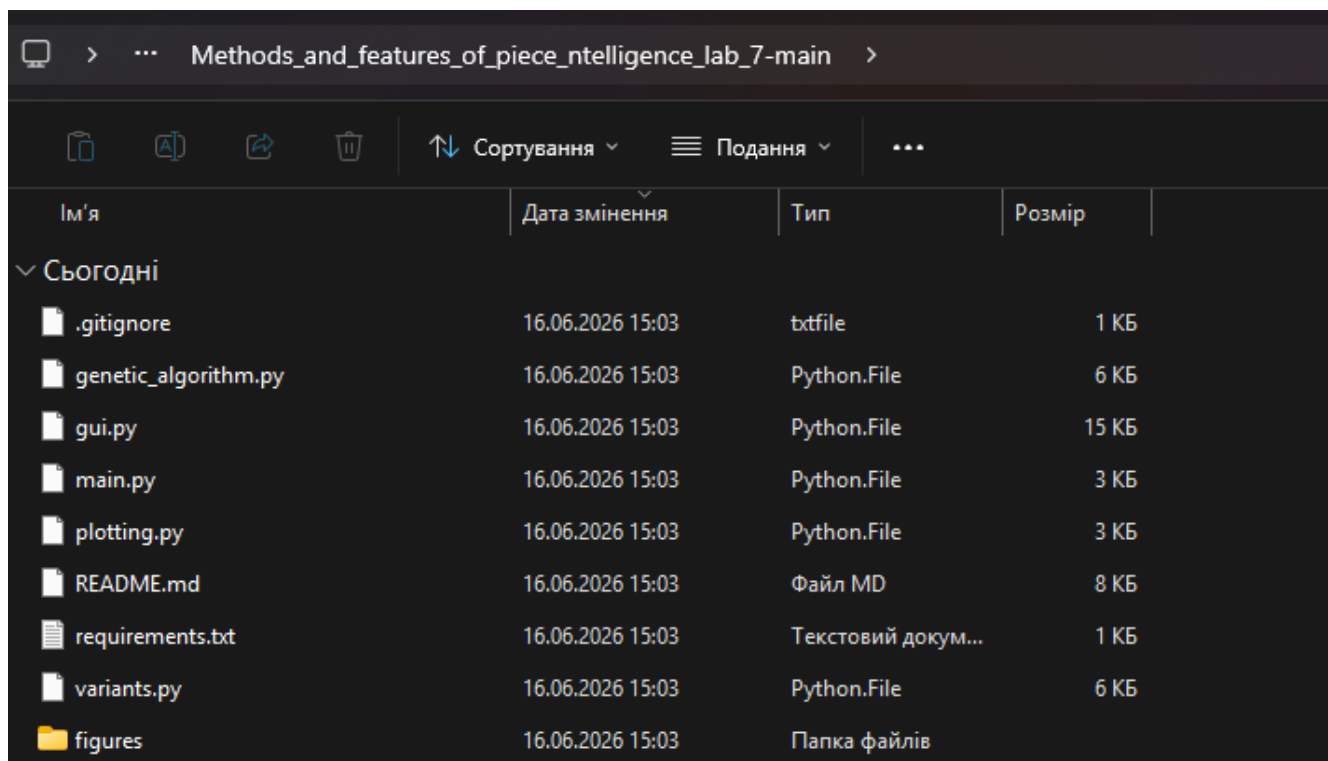
Якщо Git не встановлено: на сторінці репозиторію Code → Download ZIP, розпакувати.

Крок 2. Створити віртуальне середовище та встановити бібліотеки:

```
cd  
%USERPROFILE%\Desktop\Methods_and_features_of_piece_  
ntelligence_lab_7  
python -m venv .venv  
.venv\Scripts\activate  
pip install -r requirements.txt
```

Крок 3. Дізнатися свій номер варіанту (1...5) у викладача.

Рисунок 8.3 - Структура папки проєкту після клонування



8.2.1 Структура програмного проєкту

Файл	Призначення
variants.py	П'ять варіантів функцій, межі, режим min/max.
genetic_algorithm.py	Реалізація ГА: турнір, арифметичний кросовер, гаусова мутація, елітизм.
plotting.py	Побудова графіків для звіту.
main.py	Запуск з командного рядка (завдання 1–3).
gui.py	Графічний інтерфейс (завдання 1–4).
requirements.txt	Залежності: numpy, matplotlib.

ГА у проєкті вирішує задачу мінімізації внутрішнього cost. Для пошуку максимуму цільової функції $f(x)$ мінімізується $-f(x)$ (див. fitness_cost у variants.py).

8.2.2 Оптимізація з використанням команд (main.py)

Аналог командного режиму MATLAB ga(...). Запуск у PowerShell або CMD у папці проєкту:

```
python main.py --variant N --population 40 --
generations 120 --seed 7
```

де N - номер варіанту (1...5). Програма виводить:

- варіант і назву функції;
- режим (мінімізація / максимізація);
- межі пошуку;
- найкращу знайдену точку;
- значення цільової функції f.

Основні параметри командного рядка:

Параметр	Опис
--variant	Номер варіанту 1...5 (табл. 8.1).
--population	Розмір популяції (≥ 2).
--generations	Кількість поколінь (≥ 1).
--seed	Сід генератора випадкових чисел (відтворюваність).
--plot	Побудувати графік функції.
--plot-file	Шлях збереження графіка (наприклад figures\variantN.png).

Приклад побудови графіка (завдання 2):

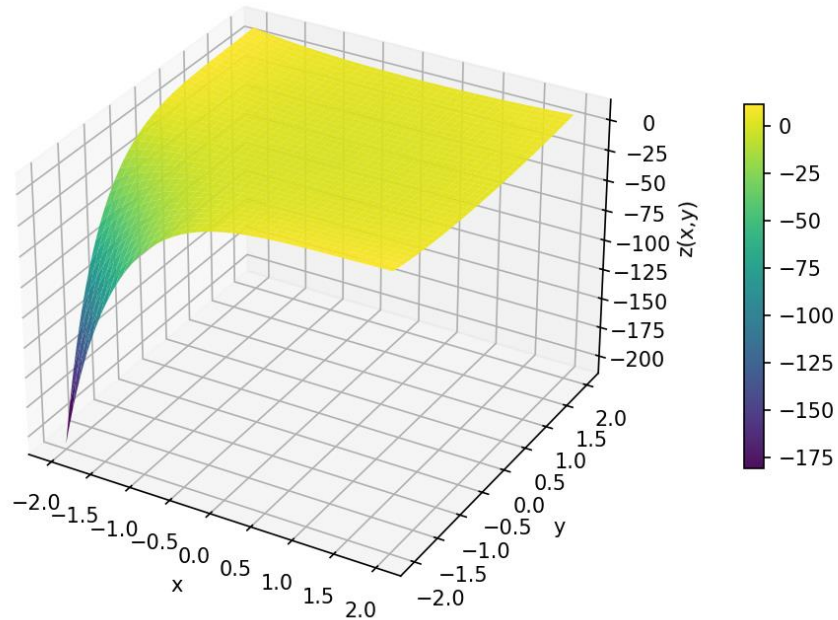
```
python main.py --variant N --plot --plot-file  
figures\variantN.png --no-show
```

Рисунок 8.4 - Результат виконання main.py у консолі (завдання 1)

```
PS C:\Users\Ярослав\Desktop\lab7ai> python main.py --variant 3 --population 40 --generations 120 --seed 7
Варіант: 3 - Двовимірний z: мінімум
Режим: мінімізація
Межі: ((-2.0, 2.0), (-2.0, 2.0))
Найкраща точка: [-2. -2.]
Значення цільової f: -210.39260013257694
PS C:\Users\Ярослав\Desktop\lab7ai> █
```

Рисунок 8.5 - Графік функції з файлу figures\variantN.png (завдання 2)

Варіант 3: Двовимірний z : мінімум

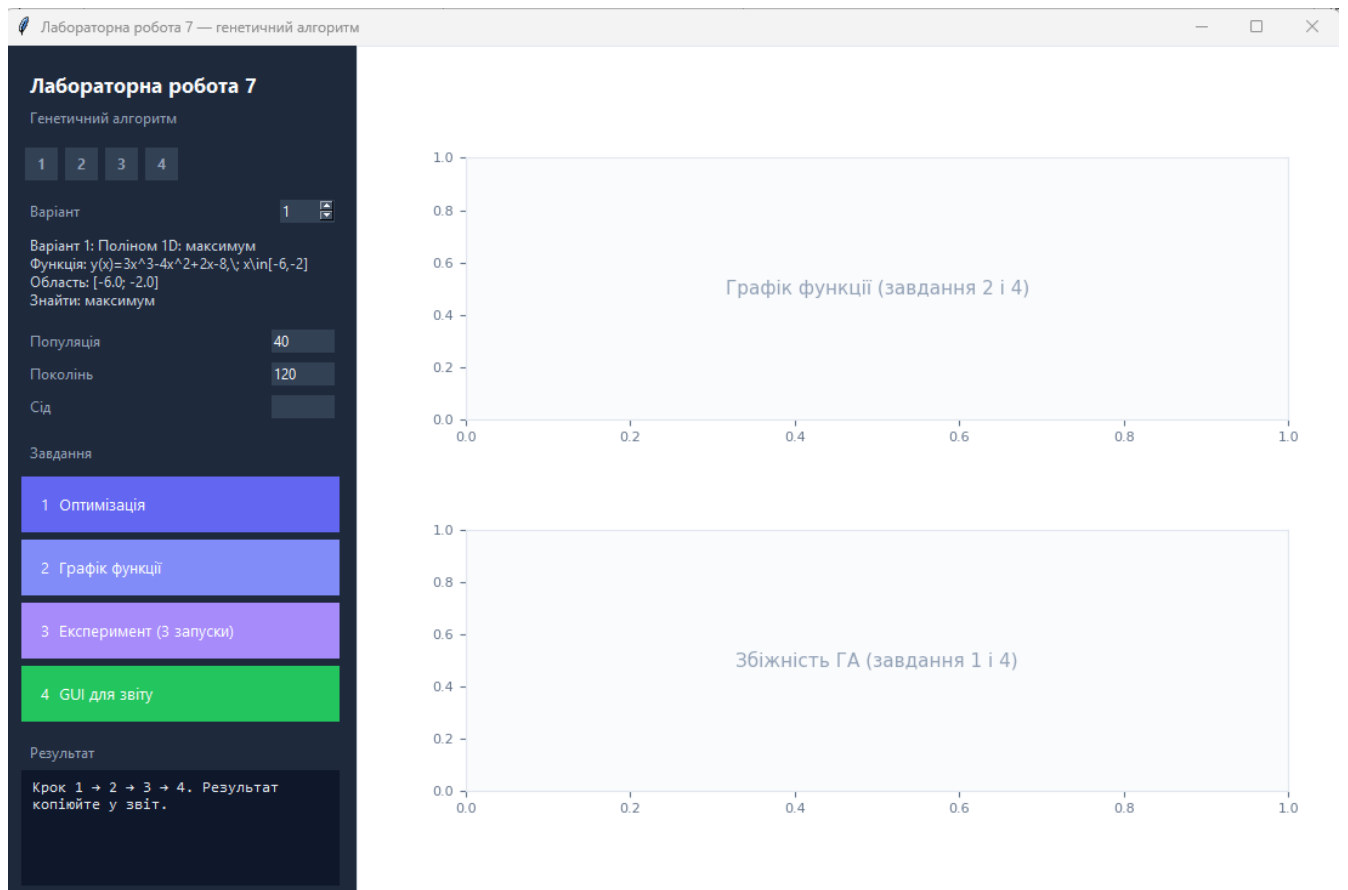


8.2.3 Оптимізація з використанням GUI (gui.py)

Аналог графічного інтерфейсу Optimization Tools / ga у MATLAB. Запуск: `python gui.py`

У вікні програми: зліва - варіант, параметри ГА, чотири кнопки завдань і поле «Результат»; справа - графіки функції та збіжності.

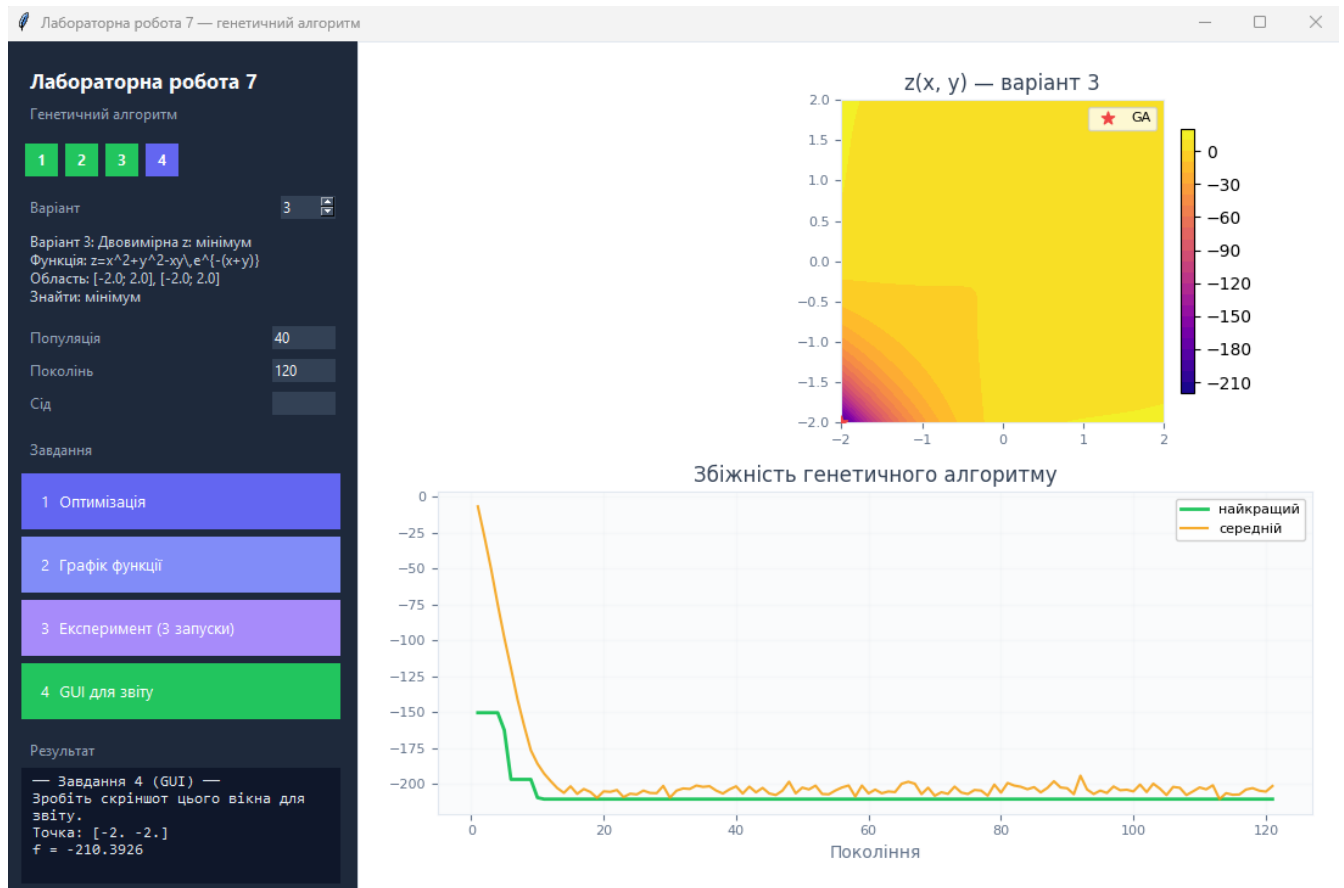
Рисунок 8.6 - Головне вікно gui.py після запуску



Кнопки відповідають завданням лабораторної:

- 12.«1 Оптимізація» - запуск ГА, вивід команди та результату.
- 13.«2 Графік функції» - побудова та збереження figures\variantN.png.
- 14.«3 Експеримент (3 запуски)» - таблиця seed / pop / gen для звіту.
- 15.«4 GUI для звіту» - повний результат + скріншот для звіту.

Рисунок 8.7 - Результат після натискання «4 GUI для звіту» (завдання 4)



8.3 Покрокове виконання лабораторної роботи

Завдання 1. Сценарій оптимізації

Оберіть свій варіант N. Запустіть оптимізацію через GUI (кнопка «1 Оптимізація») або через консоль. Запишіть у звіт команду та отримані «Найкраща точка» і f.

```
python main.py --variant N --population 40 --  
generations 120
```

Проаналізуйте графік збіжності (нижній subplot у GUI або після завдання 4): чи зменшується cost, чи є ознаки збіжності до стабільного значення.

Завдання 2. Графік функції

Побудуйте графік для свого варіанту. Для 1D - крива $y(x)$; для 2D - контурний графік або 3D-поверхня (main.py --plot). Опишіть поведінку функції: екстремуми, монотонність, особливості (полюси для вар. 2, 4).

Завдання 3. Експеримент з параметрами ГА

ГА є стохастичним: при різних seed або параметрах pop/generations результати можуть відрізнятися. У GUI кнопка «3 Експеримент (3 запуски)» виконує три запуски:

- seed = 1, population і generations - з полів GUI;
- seed = 7, ті самі population і generations;
- seed = 42, population = 80, generations = 150.

Оформіть таблицю у звіті:

seed	population	generations	знайдена точка	f
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

Напишіть 3–5 речень: чому результати збігаються або різняться; як впливають pop, generations і seed; чи є локальні екстремуми у вашій функції.

Завдання 4. Оптимізація через GUI

Натисніть «4 GUI для звіту». Переконайтеся, що на графіках відображаються функція та збіжність. Зробіть скріншот усього вікна програми та вставте у розділ «Хід роботи» звіту.

Для контролю порівняйте знайдену точку з графіком функції: чи відповідає вона очікуваному екстремуму на вашому інтервалі?

Додаток А. Параметри ГА у проєкті Python

Параметри функції `run_ga()` у `genetic_algorithm.py` (можна змінювати у коді за погодженням з викладачем):

Параметр	За замовч.	Опис
<code>population_size</code>	40	Розмір популяції N
<code>generations</code>	120	Кількість поколінь
<code>tournament_size</code>	3	Розмір турніру (селекція)
<code>crossover_prob</code>	0.85	Ймовірність кросоверу p_c
<code>mutation_prob</code>	0.15	Ймовірність мутації гена p_m
<code>mutation_sigma_scale</code>	0.05	Масштаб гаусової мутації
<code>elitism</code>	1	Кількість елітних особин
<code>seed</code>	None	Сід numpy (відтворюваність)

Приклад фрагмента коду виклику ГА у Python:

```
from genetic_algorithm import run_ga
from variants import build_variant

spec = build_variant(1)
res = run_ga(spec, population_size=40,
generations=120, seed=7)
print(res.best_phenotype, res.best_objective_value)
```

Додаток Б. Відповіді на контрольні питання

1. Основні особливості ГА.

Відповідь: Пошук із популяції точок; використання лише значень цільової функції; стохастичні правила відбору; ітеративне покращення рішення.

2. Генетичні оператори.

Відповідь: Селекція, схрещування (кросовер), мутація; часто також елітизм.

3. Критерії зупинки.

Відповідь: Ліміт поколінь, ліміт часу, поріг fitness, відсутність покращення, TolFun.

4. Схема класичного ГА.

Відповідь: Ініціалізація → оцінка → перевірка зупинки → селекція → кросовер/мутація → нове покоління → повтор.

5. Спільне використання операторів.

Відповідь: Кросовер комбінує успішні фрагменти; мутація підтримує різноманітність; баланс операторів впливає на швидкість і якість збіжності.

Перелік рекомендованої літератури

1. Рутковская Д., Пилинский М., Рутковский Л. Нейронные сети, генетические алгоритмы и нечеткие системы. М.: Горячая линия - Телеком, 2006. - 452 с.
2. Кононюк А.Ю. Нейронні мережі і генетичні алгоритми. - К.: «Корнійчук», 2008. - 446 с.

Часті помилки студентів

- Не активовано віртуальне середовище .venv перед pip install або python.
- Працюють не в папці клонованого репозиторію.
- Невірний номер варіанту (не 1...5).
- Очікують однаковий результат без однакового --seed.
- У звіті немає скріншота GUI (завдання 4).
- Не збережено графік figures\variantN.png (завдання 2).
- Не пояснили стохастичність ГА у завданні 3.